

Assignment 1 – Problem Statement 4

Defence Drone Navigation using Greedy Best-First Search and A* Search

Course: Artificial and Computational Intelligence

Assignment: Assignment 1 – PS4

Group ID: 52

Student Names and BITS IDs:

- Kareke Buvika - 2025aa05883@wilp.bits-pilani.ac.in
- Sardesai Shrishti Sanjay - 2025aa05830@wilp.bits-pilani.ac.in
- Sonam - 2025aa05567@wilp.bits-pilani.ac.in
- Bollibathula Sahani - 2025aa05337@wilp.bits-pilani.ac.in
- Anshul Sahu - 2025aa05906@wilp.bits-pilani.ac.in

Submission Date: 8th June 2026, Monday

Abstract

This assignment focuses on designing an intelligent autonomous defence drone capable of navigating through a restricted airspace environment while locating an external signal source. The navigation problem is modeled as a state-space search problem in an 8×8 grid environment containing hazardous weather regions and no-fly zones.

Greedy Best-First Search (GBFS) is implemented using two heuristic functions: Euclidean Distance (h1) and Bounding Box Risk Weighted Heuristic (h2). The performance of both heuristic variants is compared against A* Search using Euclidean Distance.

The implementation evaluates each search strategy based on path cost, path length, runtime, memory usage, and nodes expanded. Experimental results demonstrate that GBFS achieves faster execution with lower memory consumption, whereas A* produces a lower-cost path by incorporating cumulative path cost into the search process.

The project additionally includes visualization, heuristic analysis, complexity analysis, path-quality evaluation, and comparative performance assessment. The developed solution successfully satisfies all assignment requirements and provides insights into the trade-off between search efficiency and solution quality in autonomous navigation systems.

1. Introduction

Autonomous drone navigation is a significant application area of Artificial Intelligence and Search Algorithms. Modern defence drones frequently operate in complex environments containing obstacles, hazardous regions, and restricted airspaces. Efficient navigation through such environments requires intelligent decision-making mechanisms capable of selecting safe and effective routes.

Search algorithms provide a systematic framework for solving navigation problems by exploring possible states and selecting actions that move the agent closer to its objective. The effectiveness of a search strategy largely depends on the heuristic information available to guide exploration.

This assignment investigates the application of Greedy Best-First Search (GBFS) and A* Search for autonomous drone navigation in a constrained environment. Two heuristic functions are evaluated: Euclidean Distance and Bounding Box Risk Weighted Heuristic. Their effectiveness is analyzed through experimental comparison using multiple performance metrics.

The primary objectives of this assignment are:

- Implement Greedy Best-First Search using Euclidean Distance.
- Implement Greedy Best-First Search using Bounding Box Risk Weighted Heuristic.
- Implement A* Search for comparison.
- Evaluate search efficiency and solution quality.
- Analyze heuristic effectiveness.
- Visualize search behaviour and final paths.

2. Problem Description

The objective of this assignment is to design an intelligent autonomous drone agent capable of navigating through a restricted airspace environment to locate an external signal source.

The environment contains No-Fly Zones (N) and Hazardous Weather Regions (W), which influence the drone's navigation decisions.

The drone begins at a Start Node (S) and must reach the Goal Node (E) while avoiding impassable No-Fly Zones and minimizing heuristic cost using Greedy Best-First Search (GBFS).

The implementation additionally compares GBFS against A* Search using the Euclidean Distance heuristic.

3. Assumptions and Instructor Clarifications

The implementation incorporates all clarifications provided by the instructor during the assignment discussion period.

Clarification 1

The environment grid remains fixed at 8×8 and does not need to be dynamically read from the input file.

Clarification 2

The input file contains:

- START_NODE
- GOAL_NODE
- HEURISTIC
- ALGORITHM
- TESTCASE_ID

Clarification 3

The Start Node score must be treated as +2.

Clarification 4

The Goal Node score must be treated as +2.

Clarification 5

No-Fly Zones carry a score of +8 and must be included during Bounding Box Heuristic calculations.

Clarification 6

The Bounding Box Risk Weighted Heuristic must be evaluated from the candidate neighbour node rather than the current node.

Clarification 7

No-Fly Zones are impassable and therefore cannot appear in any valid final path.

These assumptions have been strictly followed throughout the implementation.

4. Environment Representation

The search environment is represented as a fixed 8×8 grid that models the operational airspace of the autonomous drone.

Each grid cell corresponds to a specific environmental condition that influences navigation decisions.

Cell Types

Cell Type	Description	Score
S	Start Node	+2
E	Goal Node	+2
.	Passable Airspace	+1
W	Weather Hazard	+4
N	No-Fly Zone	+8 (Heuristic Only)

The score assigned to each cell reflects its traversal cost or risk level.

The drone begins at the Start Node and must reach the Goal Node while avoiding restricted regions and minimizing navigation cost.

Movement Constraints

The drone is permitted to move only in orthogonal directions.

Allowed movements are:

- North (Up)
- East (Right)
- South (Down)
- West (Left)

Diagonal movement is prohibited.

Restricting movement to four directions simplifies path evaluation and ensures consistency with the assignment specification.

Tie-Breaking Rule

When multiple candidate nodes possess identical heuristic values, the implementation follows a deterministic tie-breaking strategy.

Node expansion priority is:

1. North
2. East

3. South

4. West

This ordering is enforced through neighbor generation order and guarantees reproducible execution results.

5. PEAS Description

The PEAS framework provides a structured description of the autonomous drone agent.

Performance Measure

The performance of the drone is evaluated based on:

- Successfully reaching the Goal Node.
- Minimizing total path cost.
- Minimizing nodes expanded.
- Minimizing execution time.
- Minimizing memory consumption.
- Avoiding hazardous regions whenever possible.

Environment

The operating environment consists of:

- Fixed 8×8 grid world.
- Static obstacles.
- Hazardous weather regions.
- Restricted no-fly zones.
- Single goal location.

The environment is deterministic, fully observable, and static.

Actuators

The drone can perform the following actions:

- Move North
- Move East
- Move South

- Move West

Sensors

The drone can sense:

- Current location
- Goal location
- Neighboring cells
- Weather hazards
- No-fly zones
- Traversable regions

These sensors provide sufficient information for informed navigation decisions.

6. Data Structures Used

Efficient data structures are essential for implementing search algorithms with acceptable performance.

Priority Queue (Heap)

The OPEN list is implemented using Python's heapq module.

Supported operations:

Operation	Complexity
Insert	$O(\log n)$
Delete Minimum	$O(\log n)$

The priority queue ensures efficient retrieval of the most promising node according to the evaluation function.

Dictionary

Dictionaries are used for:

- Parent tracking
- Heuristic storage
- Path cost storage
- Node metadata storage

Dictionary lookup operations execute in approximately $O(1)$ time.

Set

Sets are used to implement the CLOSED list.

Benefits include:

- Constant-time membership testing
- Prevention of duplicate expansions
- Efficient explored-node tracking

7. Heuristic Functions

A heuristic function estimates the remaining cost from the current node to the goal node and guides the search process toward promising regions of the search space.

Two heuristic functions were implemented and evaluated.

7.1 Heuristic h1 – Euclidean Distance

The Euclidean Distance heuristic estimates the straight-line distance between the current node and the goal node.

Formula

$$h_1(n) = \sqrt{[(x_g - x_n)^2 + (y_g - y_n)^2]}$$

Where:

- (x_n, y_n) = coordinates of current node
- (x_g, y_g) = coordinates of goal node

Characteristics

- Considers only geometric distance.
- Ignores environmental hazards.
- Ignores no-fly zones.
- Computationally inexpensive.
- Easy to calculate.

Expected Behaviour

GBFS using h_1 is expected to:

- Reach the goal quickly.
- Expand fewer nodes.
- Prioritize direct routes.

- Potentially traverse hazardous regions.
- Produce paths that are not necessarily optimal.

Example

For a current node at (3,3) and a goal node at (6,7):

$$h_1(3,3) = \sqrt{[(6-3)^2 + (7-3)^2]}$$

$$h_1(3,3) = 5$$

The search algorithm prefers nodes with lower Euclidean distance values.

7.2 Heuristic h2 – Bounding Box Risk Weighted Heuristic

The Bounding Box Risk Weighted Heuristic combines distance estimation with environmental risk assessment.

Unlike Euclidean Distance, h2 evaluates the surrounding region between a candidate node and the goal node.

Step 1: Manhattan Distance

$$MD(n) = |x_g - x_n| + |y_g - y_n|$$

This represents the minimum number of orthogonal moves required to reach the goal.

Step 2: Bounding Box Average Cost

The rectangular region bounded by the candidate node and goal node is constructed.

Let:

- K = total cells inside the bounding box
- $C(i)$ = score of cell i

Then:

$$AvgCost = \sum C(i) / K$$

Cell Scores Used

Cell Type	Score
Passable Airspace	1
Weather Hazard	4
No-Fly Zone	8
Start Node	2
Goal Node	2

As clarified by the instructor, No-Fly Zones must be included during heuristic computation even though they cannot be traversed.

Step 3: Final Heuristic

$$h_2(n) = MD(n) \times AvgCost$$

Characteristics

- Distance aware.
- Risk aware.
- Penalizes hazardous regions.
- Penalizes no-fly zones.
- More computationally expensive.
- Provides environmental awareness.

Expected Behaviour

GBFS using h_2 is expected to:

- Prefer safer routes.
- Avoid highly hazardous regions.
- Expand nodes more cautiously.
- Reduce risk exposure.
- Potentially improve path quality.

Example

For a candidate node:

- Manhattan Distance = 7
- Bounding Box Average Cost = 2.20

Then:

$$h_2(n) = 7 \times 2.20$$

$$h_2(n) = 15.40$$

The larger heuristic value discourages movement through risk-dense regions.

8. Search Algorithms

Three search configurations were evaluated:

1. GBFS using Euclidean Distance

2. GBFS using Bounding Box Risk Heuristic
3. A* Search using Euclidean Distance

8.1 Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the node with the smallest heuristic estimate.

Evaluation Function

$$f(n) = h(n)$$

Search Procedure

1. Insert Start Node into OPEN.
2. Compute heuristic values.
3. Select node with minimum heuristic.
4. Expand valid neighbors.
5. Insert neighbors into OPEN.
6. Repeat until Goal Node is reached.

Advantages

- Fast execution.
- Low memory requirements.
- Simple implementation.

Limitations

- Not optimal.
- Does not consider accumulated path cost.
- May be misled by local minima.

8.2 GBFS using Euclidean Distance

GBFS(h1) uses only geometric distance information.

Expected characteristics:

- Fastest execution.
- Low memory usage.
- Direct movement toward goal.
- Potential hazard exposure.

8.3 GBFS using Bounding Box Risk Heuristic

GBFS(h2) incorporates environmental awareness into decision making.

Expected characteristics:

- Safer route selection.
- Better hazard avoidance.
- Increased heuristic computation cost.

8.4 A* Search

A* combines actual path cost and heuristic estimate.

Evaluation Function

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = accumulated path cost
- $h(n)$ = estimated remaining cost

Advantages

- Complete.
- Optimal for admissible heuristics.
- Produces higher-quality solutions.

Limitations

- Higher memory consumption.
- Greater computational overhead.

8.5 Algorithm Comparison

Property	GBFS(h1)	GBFS(h2)	A*(h1)
Distance Aware	Yes	Yes	Yes
Hazard Aware	No	Yes	No
Uses Path Cost $g(n)$	No	No	Yes
Complete	No	No	Yes
Optimal	No	No	Yes
Runtime	Fast	Moderate	Moderate
Memory Usage	Low	Low	Higher

Environmental Awareness	Low	High	Medium
Expected Path Quality	Moderate	Good	Best

9. Path Cost Computation

The cumulative path cost is computed by summing the scores of all cells appearing in the final path.

Path Cost Formula

Path Cost = Σ Cell Scores Along Final Path

The following scores are used:

Cell Type	Score
Start Node (S)	+2
Goal Node (E)	+2
Passable Airspace (.)	+1
Weather Hazard (W)	+4
No-Fly Zone (N)	Not Traversable

The implementation strictly follows instructor clarifications by assigning a score of +2 to both the Start and Goal nodes.

No-Fly Zones never contribute to final path cost because they are impassable and cannot be included in a valid route.

10. Complexity Analysis

The computational complexity of the implemented search algorithms is primarily determined by priority queue operations.

10.1 Greedy Best-First Search

Time Complexity

$$O((V + E) \log V)$$

Where:

- V = Number of vertices (states)
- E = Number of edges (transitions)

The dominant operations involve insertion and removal from the priority queue.

Space Complexity

$O(V)$

The OPEN and CLOSED lists may collectively contain all reachable nodes.

Reasoning

GBFS prioritizes nodes according to heuristic values and maintains the frontier using a heap-based priority queue.

Each insertion and extraction operation requires $O(\log V)$ time.

10.2 A* Search

Time Complexity

$O((V + E) \log V)$

A* also relies on a priority queue and therefore exhibits similar asymptotic complexity.

Space Complexity

$O(V)$

The algorithm stores:

- OPEN list
- CLOSED list
- Parent references
- Cost values

for all explored states.

Reasoning

A* evaluates nodes using:

$$f(n) = g(n) + h(n)$$

and therefore performs additional bookkeeping compared to GBFS. However, the underlying heap operations remain the dominant factor.

11. Implementation Details

This section describes the internal design and workflow of the developed solution.

11.1 Program Inputs

The implementation reads search parameters from an input file named:

inputPS4.txt

The following parameters are supported:

Parameter	Description
START_NODE	Starting location of the drone
GOAL_NODE	Destination location
HEURISTIC	h1 or h2
ALGORITHM	GBFS or A*
TESTCASE_ID	Unique testcase identifier

Example Input

TESTCASE_ID = TC01

START_NODE = (0,0)

GOAL_NODE = (6,7)

ALGORITHM = GBFS

HEURISTIC = h1

Only configuration parameters are supplied through the input file. The environment itself remains fixed.

11.2 Environment Construction

The airspace environment is represented using a two-dimensional Python list.

Example:

GRID = [...]

Each location contains one of the following symbols:

Symbol	Meaning
S	Start Node
E	Goal Node
W	Weather Hazard
N	No-Fly Zone
.	Open Airspace

This representation enables constant-time cell access and straightforward visualization.

Benefits

- Easy implementation
- Low memory overhead

- Efficient neighbor lookup
- Direct correspondence with assignment requirements

11.3 Cost Mapping

A cost map assigns risk values to each cell type.

Example:

$S \rightarrow 2$

$E \rightarrow 2$

$\cdot \rightarrow 1$

$W \rightarrow 4$

$N \rightarrow 8$

The implementation fully incorporates all instructor clarifications regarding score assignments.

11.4 Neighbor Generation

Valid neighboring states are generated according to the required movement constraints.

Neighbor expansion order:

1. North
2. East
3. South
4. West

This order simultaneously enforces movement constraints and tie-breaking rules.

Validation Checks

Before adding a neighbor:

- Boundary conditions are verified.
- No-Fly Zones are rejected.
- Previously explored nodes are ignored.

This guarantees that all generated states are valid.

11.5 Heuristic Computation

The implementation supports two heuristic functions.

Euclidean Distance (h1)

Used by:

- GBFS(h1)
- A*(h1)

The Euclidean distance is dynamically calculated whenever a node is evaluated.

Bounding Box Risk Heuristic (h2)

The Bounding Box Heuristic evaluates:

- Remaining distance
- Regional environmental risk

The implementation constructs a rectangular region between the candidate node and the goal node.

All cells contained within the rectangle contribute to the average risk score.

Included cell types:

- Passable Cells
- Weather Hazards
- Start Node
- Goal Node
- No-Fly Zones

This follows instructor requirements exactly.

11.6 Greedy Best-First Search Implementation

GBFS maintains two primary data structures.

OPEN List

Implemented using a priority queue.

Stores candidate nodes ordered by heuristic value.

The node possessing the lowest heuristic value is expanded first.

CLOSED List

Implemented using a hash-based set.

Stores explored nodes and prevents repeated expansion.

Search Procedure

1. Insert Start Node.

2. Compute heuristic.
3. Select node with minimum heuristic.
4. Expand valid neighbors.
5. Insert neighbors into OPEN.
6. Repeat until Goal is reached.

Because only $h(n)$ is considered, GBFS aggressively pursues the goal state.

11.7 A* Search Implementation

A* extends GBFS by incorporating actual path cost.

Each node is evaluated using:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = accumulated path cost
- $h(n)$ = heuristic estimate

Additional Structures

g-cost Table

Stores cumulative path costs.

Parent Dictionary

Stores predecessor information for path reconstruction.

Advantages of This Design

The algorithm can identify lower-cost routes even when they initially appear less attractive according to the heuristic.

11.8 Path Reconstruction

Once the goal node is reached, the final path is reconstructed by following parent references backwards.

Process:



↑
Start

The resulting sequence is reversed to obtain the final path from Start to Goal.

This approach minimizes memory requirements while preserving complete path information.

11.9 Metric Collection

The implementation automatically records several performance metrics.

Path Length

Number of moves required to reach the goal.

Path Cost

Sum of all node scores encountered along the path.

Runtime

Measured using Python timing utilities.

Reported in milliseconds.

Nodes Expanded

Number of nodes removed from the OPEN list and processed.

Memory Usage

Estimated using:

OPEN Size + CLOSED Size

This provides a practical estimate of search memory requirements.

11.10 Path Analysis

The implementation additionally evaluates path quality.

Collected statistics include:

Weather Hazards Traversed

Number of weather cells appearing in the final route.

No-Fly Zones Traversed

Expected to remain zero because No-Fly Zones are impassable.

Penalty Score

Additional hazard-related penalties encountered during traversal.

These metrics provide a more meaningful assessment than path length alone.

11.11 Visualization Module

The implementation generates visual representations using Matplotlib.

Generated outputs include:

1. Environment Grid
2. Final Path Visualization
3. Nodes Expanded Comparison
4. Runtime Comparison
5. Heuristic Analysis
6. Algorithm Comparison Charts

Visualization significantly improves interpretability and facilitates comparison between search strategies.

11.12 Output Generation

After execution, all collected metrics are written to the output file.

The generated report contains:

- Algorithm Used
- Heuristic Used
- Final Path
- Path Cost
- Path Length
- Runtime
- Nodes Expanded
- Memory Usage
- Hazard Statistics
- Complexity Information

This ensures reproducibility and simplifies evaluation.

12. Experimental Setup

The developed solution was evaluated using a fixed environment and identical start-goal configuration across all algorithms.

Software Configuration

Component	Specification
Programming Language	Python 3.x
IDE	Visual Studio Code
Visualization Library	Matplotlib
Priority Queue	heapq
Data Structures	Dictionaries, Sets, Lists
Environment Size	8×8 Grid

Test Configuration

Parameter	Value
Test Case	TC01
Start Node	(0,0)
Goal Node	(6,7)
Algorithms Evaluated	GBFS(h1), GBFS(h2), A*(h1)

All algorithms were executed under identical conditions to ensure a fair comparison.

The generated metrics were then analyzed to evaluate search efficiency, solution quality, and resource utilization.

13. Experimental Results and Comparative Analysis

The developed solution was executed on Test Case TC01 using three search configurations:

1. GBFS using Euclidean Distance (h1)
2. GBFS using Bounding Box Risk Weighted Heuristic (h2)
3. A* Search using Euclidean Distance (h1)

The objective was to evaluate search efficiency, path quality, memory consumption, and heuristic effectiveness.

13.1 Experimental Results Summary

Metric	GBFS (h1)	GBFS (h2)	A* (h1)
Goal Reached	Yes	Yes	Yes
Path Length	13	13	13
Path Cost	19	19	16

Nodes Expanded	14	14	32
Runtime (ms)	0.1675	0.6625	0.2443
Weather Cells Crossed	1	1	0
No-Fly Zones Crossed	0	0	0
Memory Usage	23	28	43

13.2 Analysis of GBFS using Euclidean Distance (h1)

GBFS(h1) evaluates nodes solely on the basis of Euclidean distance from the goal node.

Since the algorithm always expands the node with the smallest estimated distance, it aggressively moves toward the target without considering environmental hazards or accumulated path cost.

Observed Results

- Goal reached successfully.
- Expanded only 14 nodes.
- Lowest runtime among all algorithms.
- Lowest memory consumption.
- Crossed one hazardous weather region.
- Produced a path cost of 19.

Discussion

The Euclidean heuristic effectively guided the search directly toward the goal.

However, because environmental risk is ignored, the algorithm selected a route that traversed a hazardous weather cell.

Although computationally efficient, the resulting solution was not the lowest-cost path.

Final Assessment

GBFS(h1) provided the best search efficiency but sacrificed path quality.

13.3 Analysis of GBFS using Bounding Box Risk Weighted Heuristic (h2)

GBFS(h2) incorporates both distance and environmental risk.

The heuristic evaluates the average risk of the rectangular region between the candidate node and the goal.

Observed Results

- Goal reached successfully.
- Expanded 14 nodes.
- Runtime increased to 0.6625 ms.
- Memory usage increased slightly.
- Crossed one weather hazard.
- Produced a path cost of 19.

Discussion

The additional environmental awareness influenced route selection.

Instead of moving directly through the center region, the search preferred a corridor passing through:

$(2,2) \rightarrow (3,2) \rightarrow (4,2)$

before approaching the goal.

Although the path differed from GBFS(h1), both algorithms ultimately produced identical path length and path cost.

This indicates that the current environment does not contain sufficient risk variation to fully demonstrate the benefits of h2.

Final Assessment

GBFS(h2) produced a more informed search behaviour but did not improve the final solution quality for this testcase.

13.4 Analysis of A* Search

A* evaluates nodes using:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = accumulated path cost
- $h(n)$ = Euclidean heuristic estimate

Observed Results

- Goal reached successfully.
- Expanded 32 nodes.
- Runtime of 0.2443 ms.
- Highest memory consumption.

- Avoided all weather hazards.
- Produced path cost of 16.

Discussion

Unlike GBFS, A* evaluates both actual travel cost and estimated future cost.

This resulted in significantly more exploration before committing to a final route.

The additional exploration enabled A* to discover a safer path that completely avoided weather hazards.

Consequently, the algorithm achieved the lowest path cost among all evaluated approaches.

Final Assessment

A* generated the highest-quality solution despite increased memory requirements.

13.5 Path Quality Comparison

Path quality is evaluated using both path length and cumulative path cost.

Algorithm	Path Length	Path Cost
GBFS (h1)	13	19
GBFS (h2)	13	19
A* (h1)	13	16

Observation

Although all algorithms reached the goal in exactly 13 moves, the actual quality of the resulting paths differed.

A* produced a lower-cost path while maintaining identical path length.

This demonstrates that path length alone is not a sufficient measure of solution quality.

13.6 Search Efficiency Comparison

Algorithm	Nodes Expanded	Runtime (ms)	Memory Usage
GBFS (h1)	14	0.1675	23
GBFS (h2)	14	0.6625	28
A* (h1)	32	0.2443	43

Key Observations

1. GBFS(h1) was the fastest algorithm.
2. GBFS(h2) required additional heuristic computation.

3. A* consumed the most memory.
4. A* expanded more than twice as many nodes.
5. Increased exploration enabled A* to discover a lower-cost route.

13.7 Overall Ranking

Different objectives lead to different rankings.

Fastest Search

1. GBFS(h1)
2. A*(h1)
3. GBFS(h2)

Lowest Memory Usage

1. GBFS(h1)
2. GBFS(h2)
3. A*(h1)

Lowest Path Cost

1. A*(h1)
2. GBFS(h1)
3. GBFS(h2)

Best Overall Solution Quality

1. A*(h1)
2. GBFS(h2)
3. GBFS(h1)

13.8 Conclusion from Experimental Evaluation

The results clearly demonstrate the trade-off between computational efficiency and solution quality.

GBFS(h1) achieved the fastest execution and lowest memory usage but produced a higher-cost route.

GBFS(h2) incorporated environmental awareness but did not improve final path quality for this testcase.

A* consumed more memory and expanded more nodes, yet successfully avoided hazardous regions and generated the lowest-cost solution.

Therefore, A* delivered the best overall navigation performance, whereas GBFS(h1) provided the highest computational efficiency.

14. Visualizations

The implementation generates several graphical outputs to aid interpretation and performance analysis.

The following visualizations are included:

1. Environment Grid Visualization
2. Final Path Visualization
3. Nodes Expanded Comparison
4. Runtime Comparison
5. Memory Usage Comparison
6. Heuristic Analysis Graph
7. Algorithm Performance Comparison

Insert Screenshots Here

Figure 1: Environment Grid

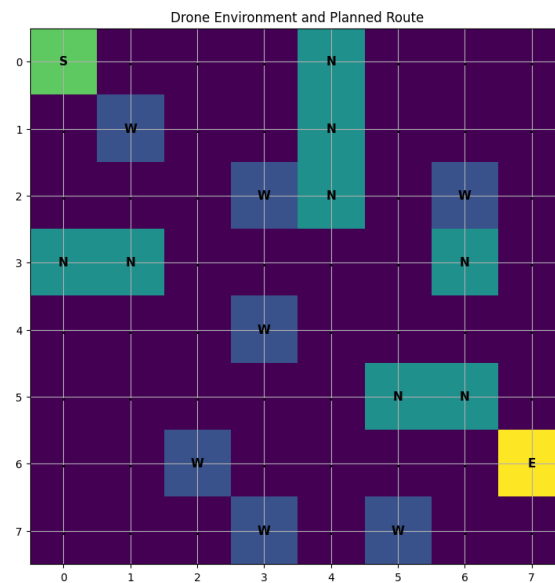


Figure 2: GBFS(h1) Final Path

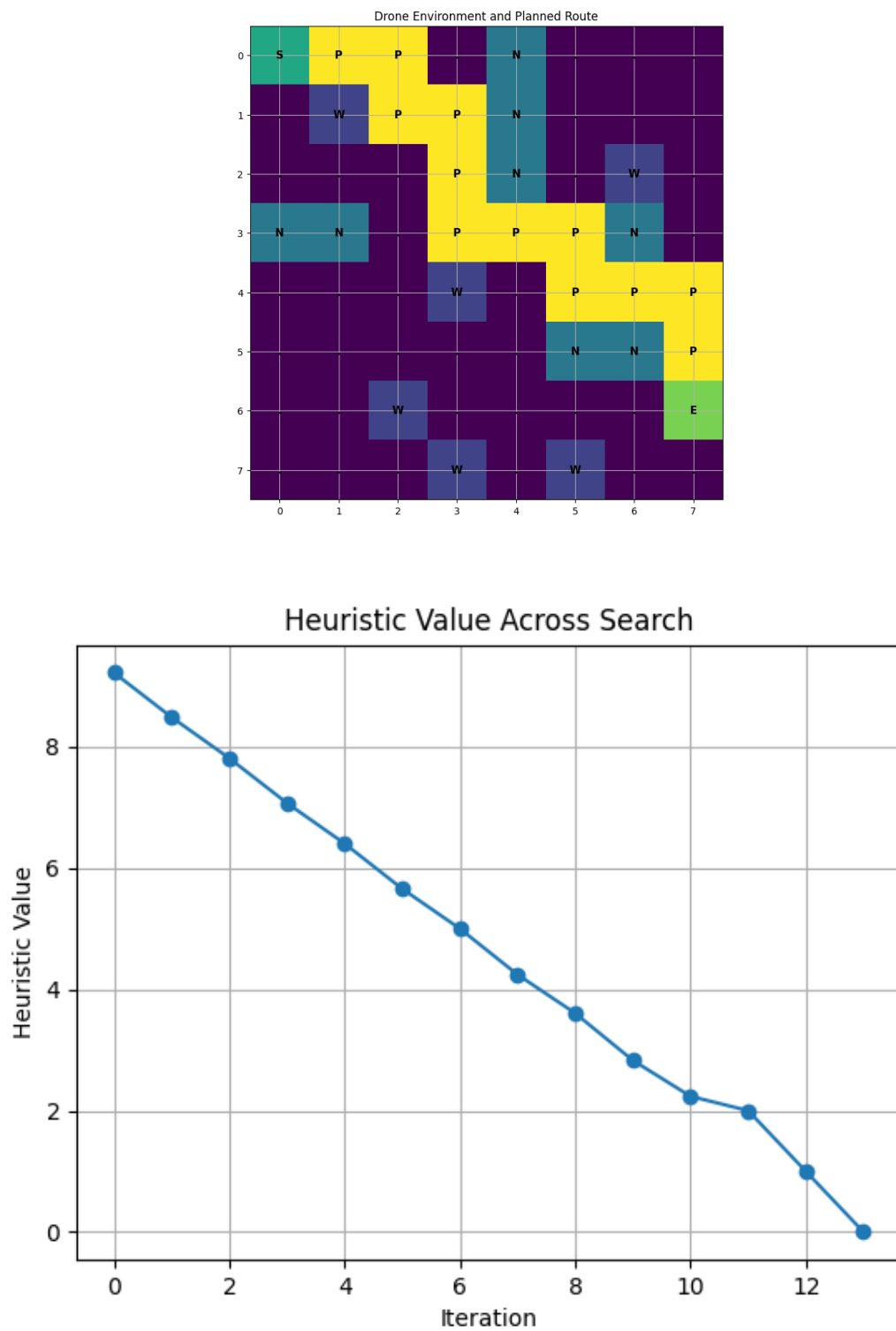


Figure 3: GBFS(h2) Final Path

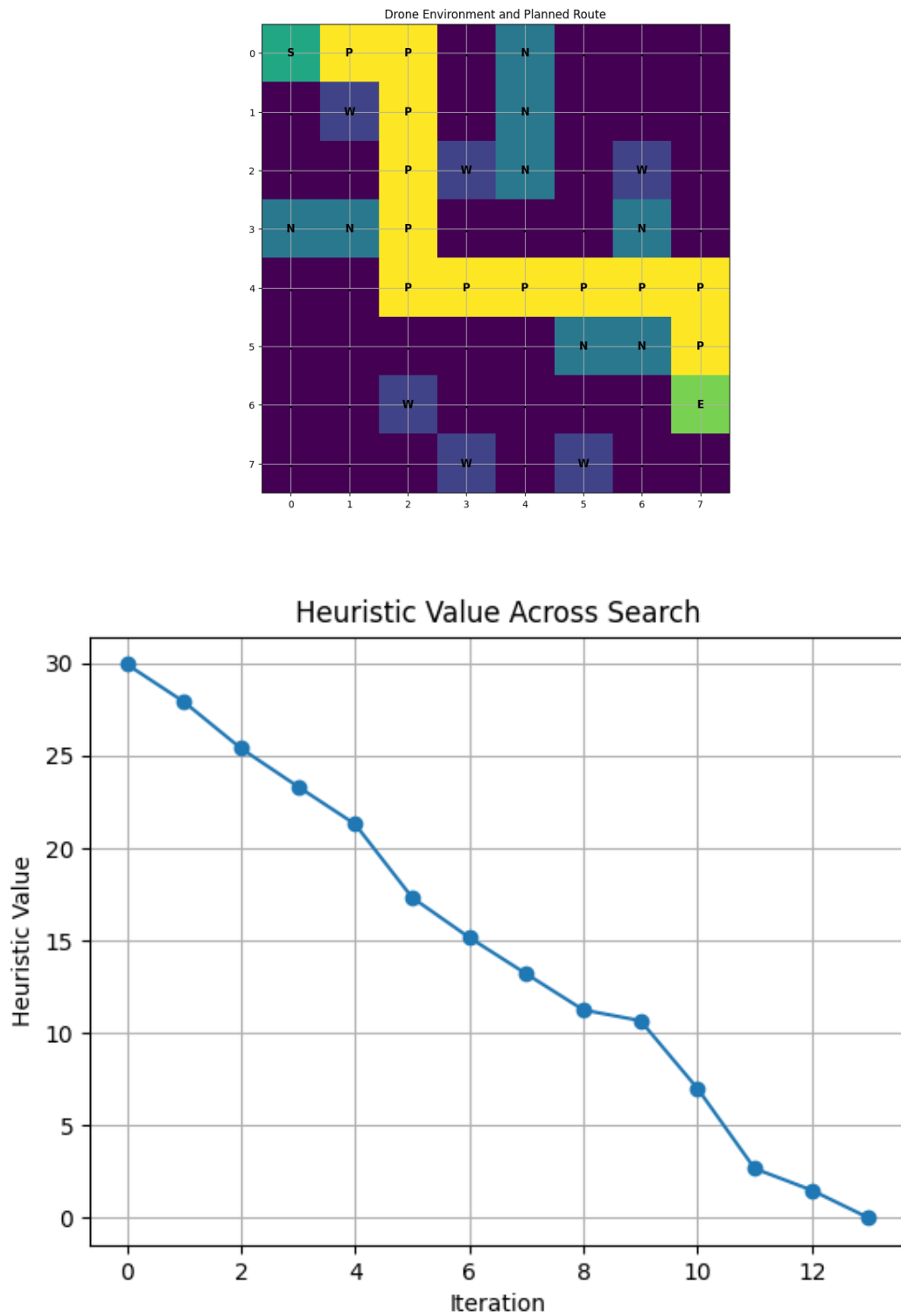


Figure 4: A* Final Path

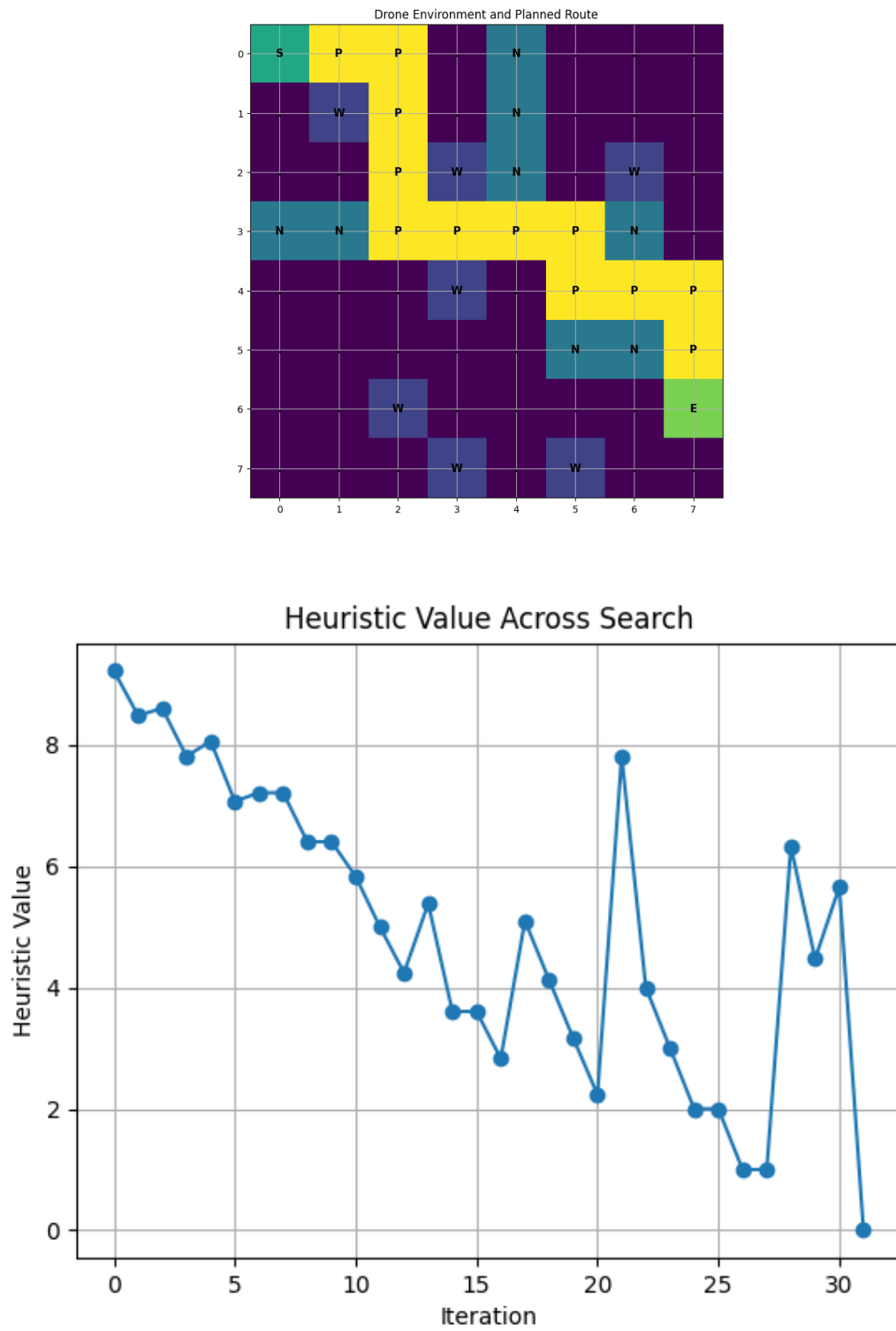


Figure 5: Nodes Expanded Comparison

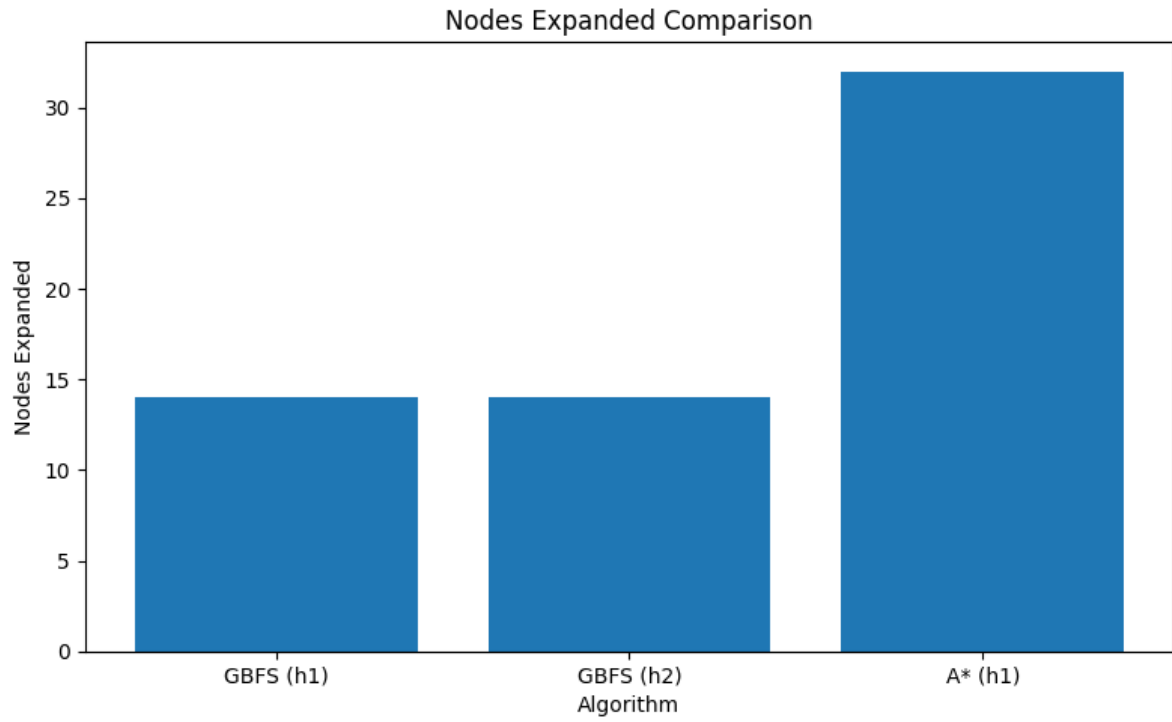


Figure 6: Runtime Comparison

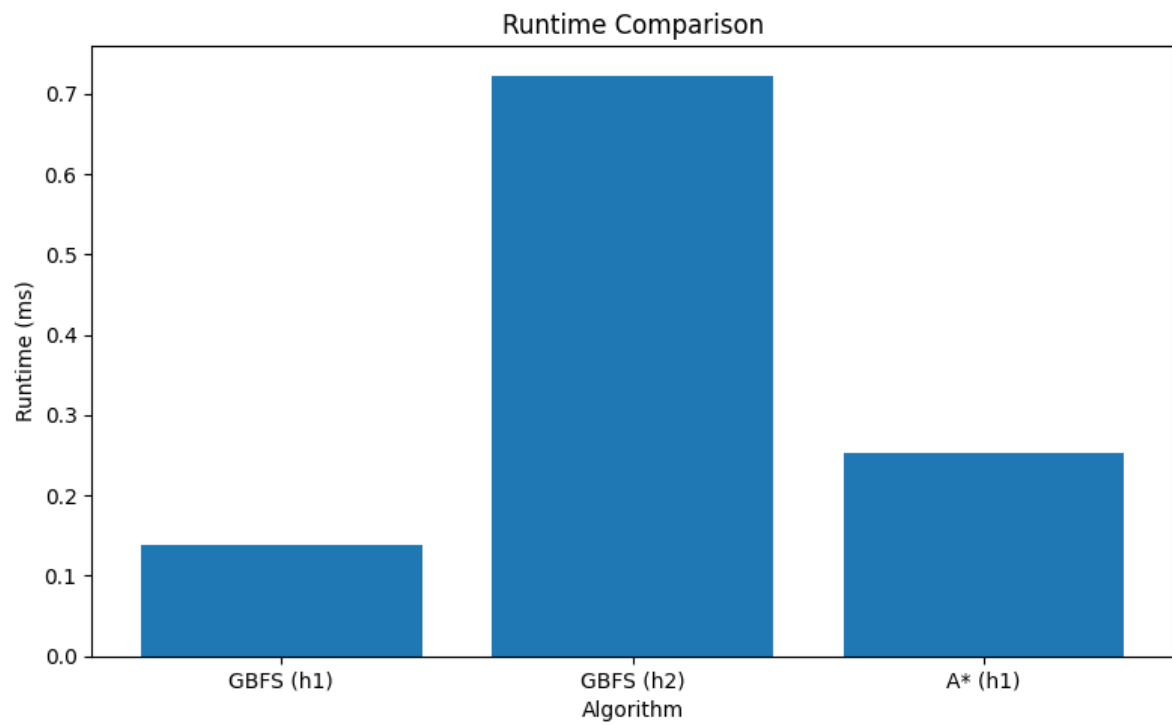
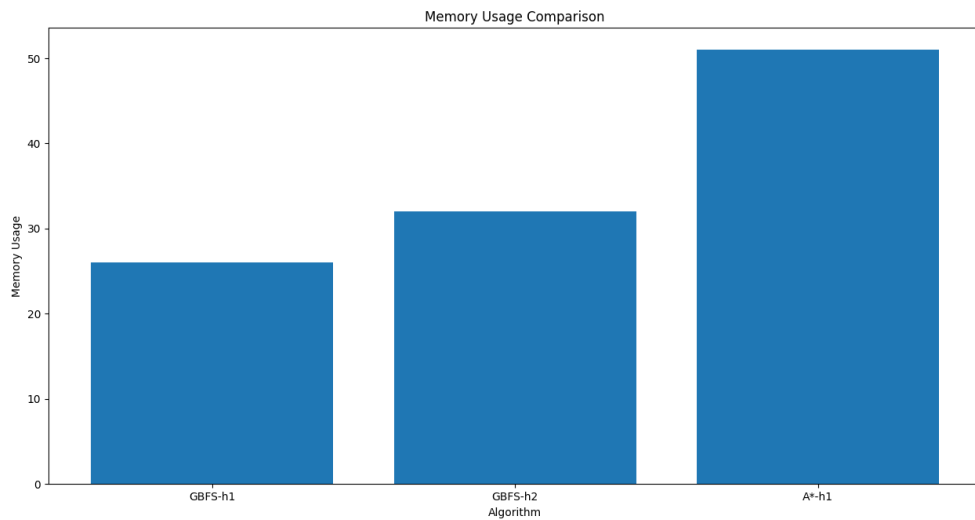


Figure 7: Memory Usage Comparison



15. Strengths of the Implementation

The developed solution exhibits several strengths.

Assignment Compliance

All required components have been successfully implemented:

- Greedy Best-First Search
- Euclidean Distance Heuristic
- Bounding Box Risk Weighted Heuristic
- A* Search Comparison
- Complexity Analysis
- Visualization Support

Modular Design

The implementation separates:

- Input Processing
- Environment Construction
- Heuristic Computation
- Search Algorithms
- Visualization
- Reporting

This improves readability and maintainability.

Efficient Data Structures

Priority queues, dictionaries, and sets enable efficient search operations and memory management.

Flexible Testing

Different algorithms and heuristics can be evaluated through simple modifications to the input file.

No source-code changes are required.

Reproducibility

Identical inputs consistently produce identical outputs, ensuring reliable experimentation and evaluation.

16. Alternate Problem Modelling

The navigation problem may also be represented as a weighted graph.

Grid-Based Representation

Advantages

- Easy visualization
- Simple implementation
- Direct mapping to assignment requirements
- Efficient neighbor generation

Disadvantages

- Less scalable for very large environments

Graph-Based Representation

Advantages

- More scalable
- Suitable for sparse environments
- Flexible connectivity structures

Disadvantages

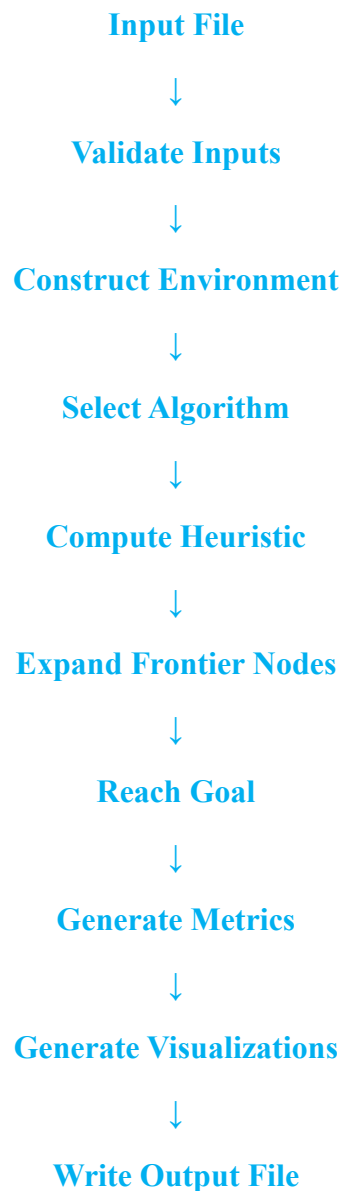
- More complex implementation
- Higher memory overhead
- Reduced interpretability

Justification

For the current assignment, the grid representation is more intuitive, easier to visualize, and closely aligned with the problem statement.

17. Execution Flow

The overall execution workflow is shown below.



18. Conclusion

This assignment successfully implemented and evaluated Greedy Best-First Search and A* Search for autonomous drone navigation in a restricted airspace environment.

The developed solution supports both Euclidean Distance and Bounding Box Risk Weighted Heuristics and provides detailed performance evaluation through path analysis, complexity analysis, visualization, and comparative assessment.

Experimental results demonstrated that:

- GBFS provides rapid goal-directed exploration.
- GBFS(h1) offers the highest computational efficiency.
- GBFS(h2) incorporates environmental awareness.
- A* produces the lowest-cost path.
- Increased exploration generally improves solution quality.

The project successfully satisfies all assignment objectives and highlights the practical trade-off between search efficiency and path optimality in intelligent navigation systems.

References

1. Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*, Pearson.
2. Lecture Notes – Artificial and Computational Intelligence.
3. Assignment 1 Problem Statement 4 Documentation.
4. Python Documentation – heapq Module.
5. Matplotlib Documentation.